A Treatise on Model Oriented Programming Languages

Thirteen years ago I developed to my knowledge the first and currently the only fully featured model oriented programming language and IDE. I utilized this technology (Mo+) to great effect on my own and workplace enterprise projects, but failed to sell the ideas to a wider audience. Six years ago I left my software engineering career in favor of wielding an ax and building Viking and Anglo Saxon ships in Norway, UK and other future places in Scandinavia. Even so, I still think about model oriented programming from time to time and its potential.

The purpose of this article is not about the technology I developed, but to inspire research, development, and use of model oriented languages. Amongst other things, I would love to make a contribution to any research going on at a university or business. Please let me know if you are aware of any research and development of model oriented programming languages going on out there!

I firmly believe that the family of model oriented programming languages has great future potential, even and maybe even especially in the current world of AI. I hope this article will inspire a deeper look into such potential.

Below, I will outline some principles and features I think are key to make the most of model oriented programming languages. Unless otherwise noted, these principles and features are not pure theory, but were actually utilized in enterprise systems over several years. There are many concepts to go over here. If the earlier sections get a little dry and tedious, at least skip down to the language features section.

What is a Model?

There are of course many kinds of models and many ways to describe a model. At its core, the purpose of a model is to concisely represent something else that is larger or more complex. For model oriented programming, we will need an electronic definition of a model, and the following definition can be used:

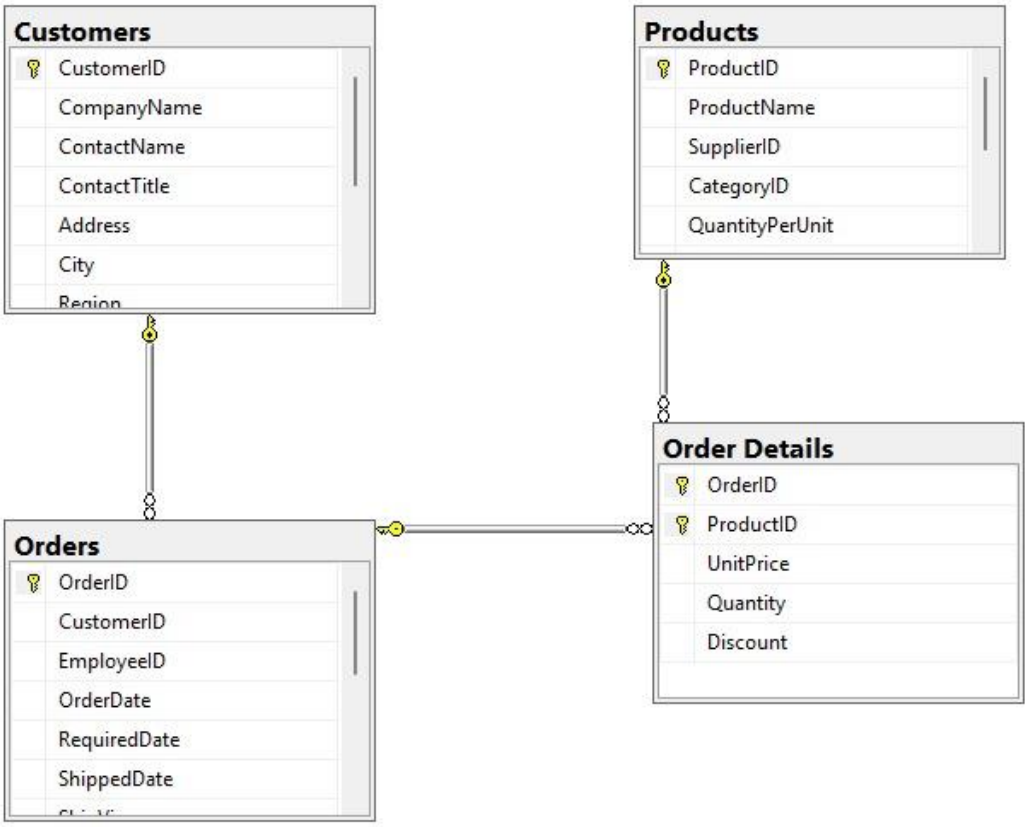
Model – A model can be described in terms of **structure** and **data**. The structure defines the rules or “schema” of a model and is always hierarchical. A particular model is populated with data that conforms to the model structure.

Model Structure and Data

A model structure can simply be defined in terms of ***nodes*** and ***attributes***:

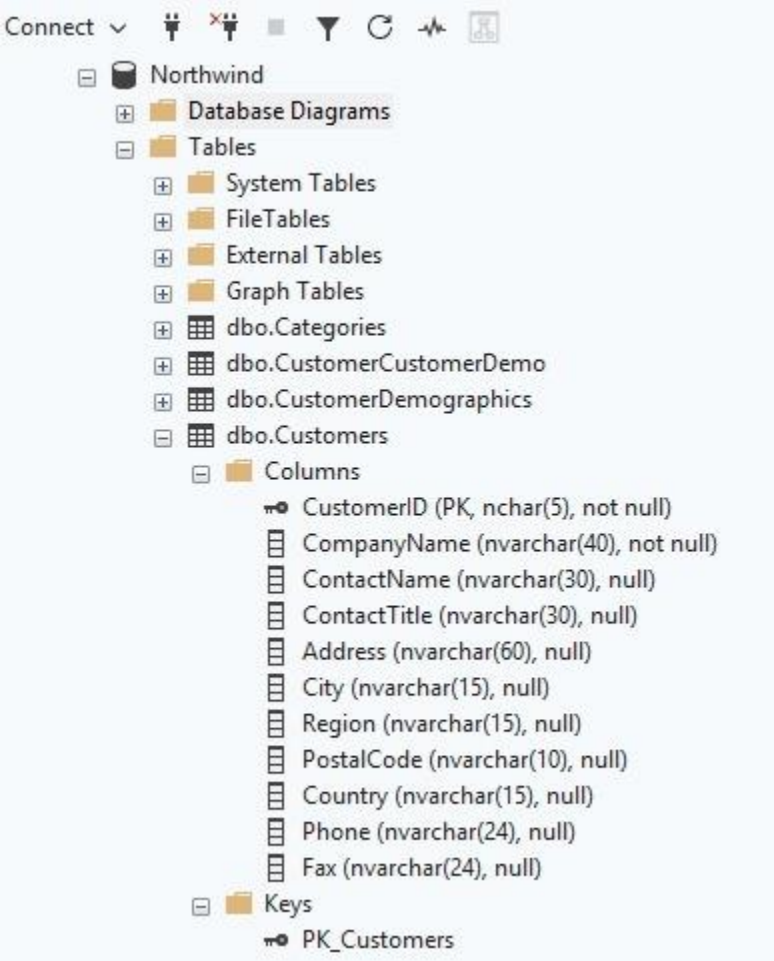
- A model structure has a single root node.
- The root node can have any number of child nodes. Any node in turn can have any number of child nodes to form a hierarchy of any given depth.
- Nodes are identified with at least a name. In addition, nodes can be further described with any number of attributes.

You may not believe me yet that model structures can always be hierarchical with a single root node. Let's use the following diagram of a portion of the Northwind database as an example:



You may say "this is not hierarchical!!". But, what are we looking at here? In our context, it is not model structure, but model data! You can say that the Northwind database schema is an example of populating a standard relational database structure with data. Rows of data in database tables is run time data, and not what I am talking about here. The schema is an example of model data.

You can see the hierarchical structure in a relational database IDE as the schema can be displayed in a tree view:



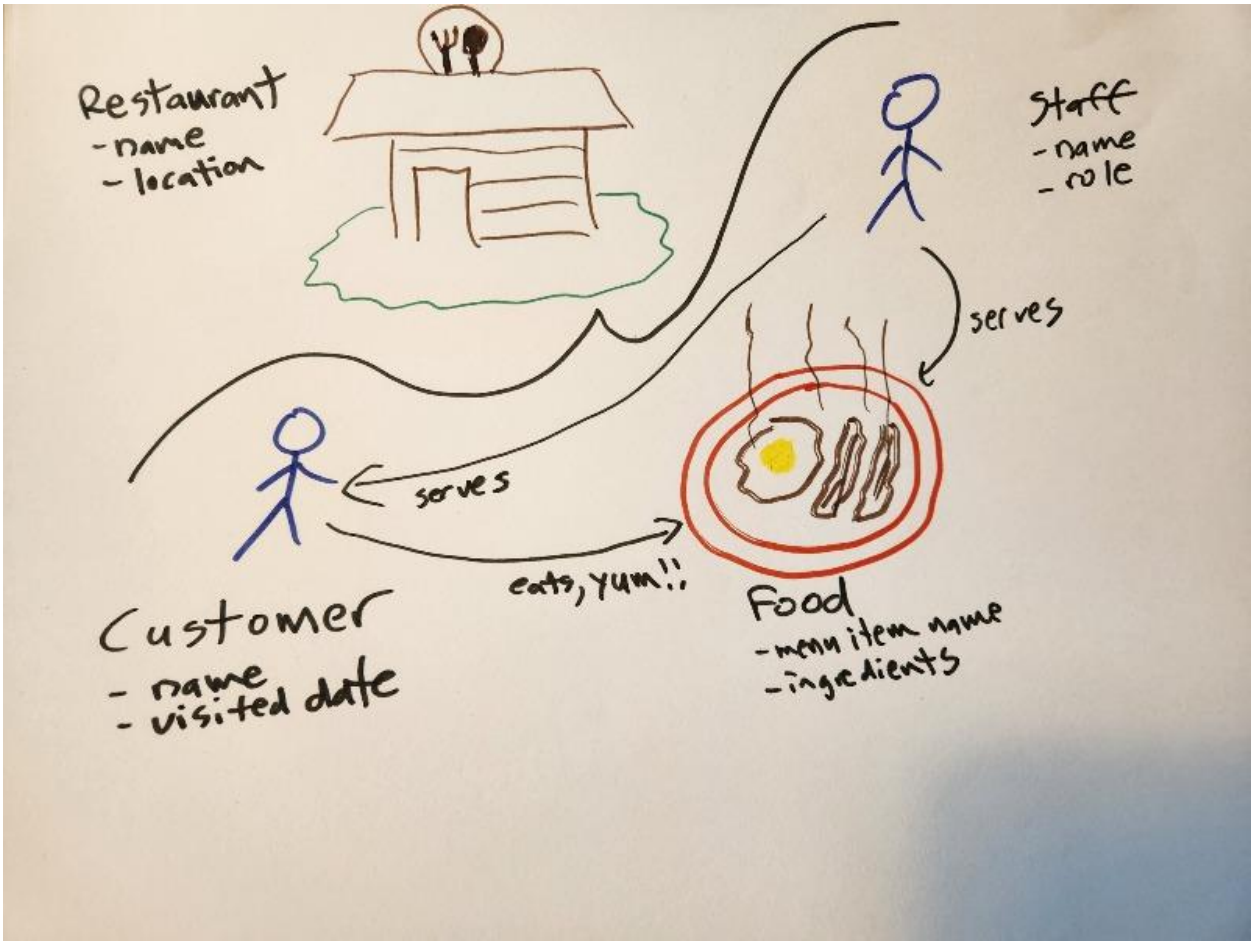
So the model structure of a relational database includes nodes such as database, table, column, and key that fall neatly into a hierarchy. You may say “wait, the hierarchy is broken, as foreign keys refer to tables!”. This is handled very simply as follows:

- An attribute of a model structure node can refer to another node in the model structure.

With the above rule, we can have very complex model structures that fall neatly into a hierarchy. I would add that in a good model structure definition, nodes are defined at the highest level where they can stand on their own.

Example Scenario and Model

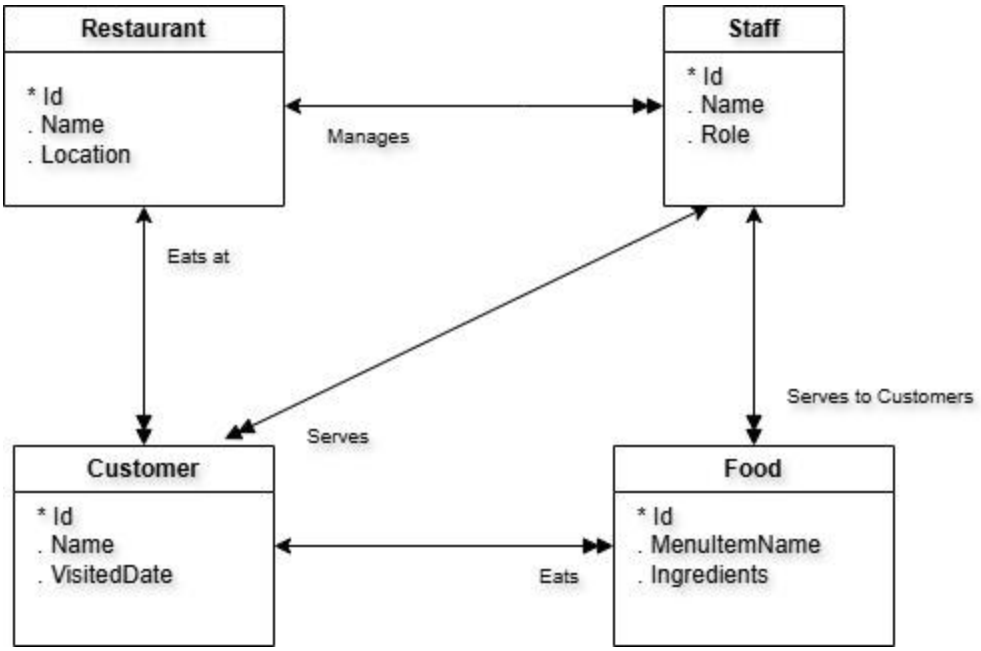
For the remainder of this article we will use a very simplified restaurant scenario with which we will model in various ways and explain how we could program to the scenario with model oriented programming languages.



The simplified restaurant scenario is described as such:

- We want to manage a number of restaurants and need to track their name and location.
- A restaurant has any number of customers and we want to track customers by name and when they visit (we are ignoring repeat customers in this simple scenario).
- A restaurant has any number of staff, and we want to track staff names and their role.
- Customer eat food and we want to track what menu items they order.
- Staff serve customers and food and we want to track which customers and menu items they serve.

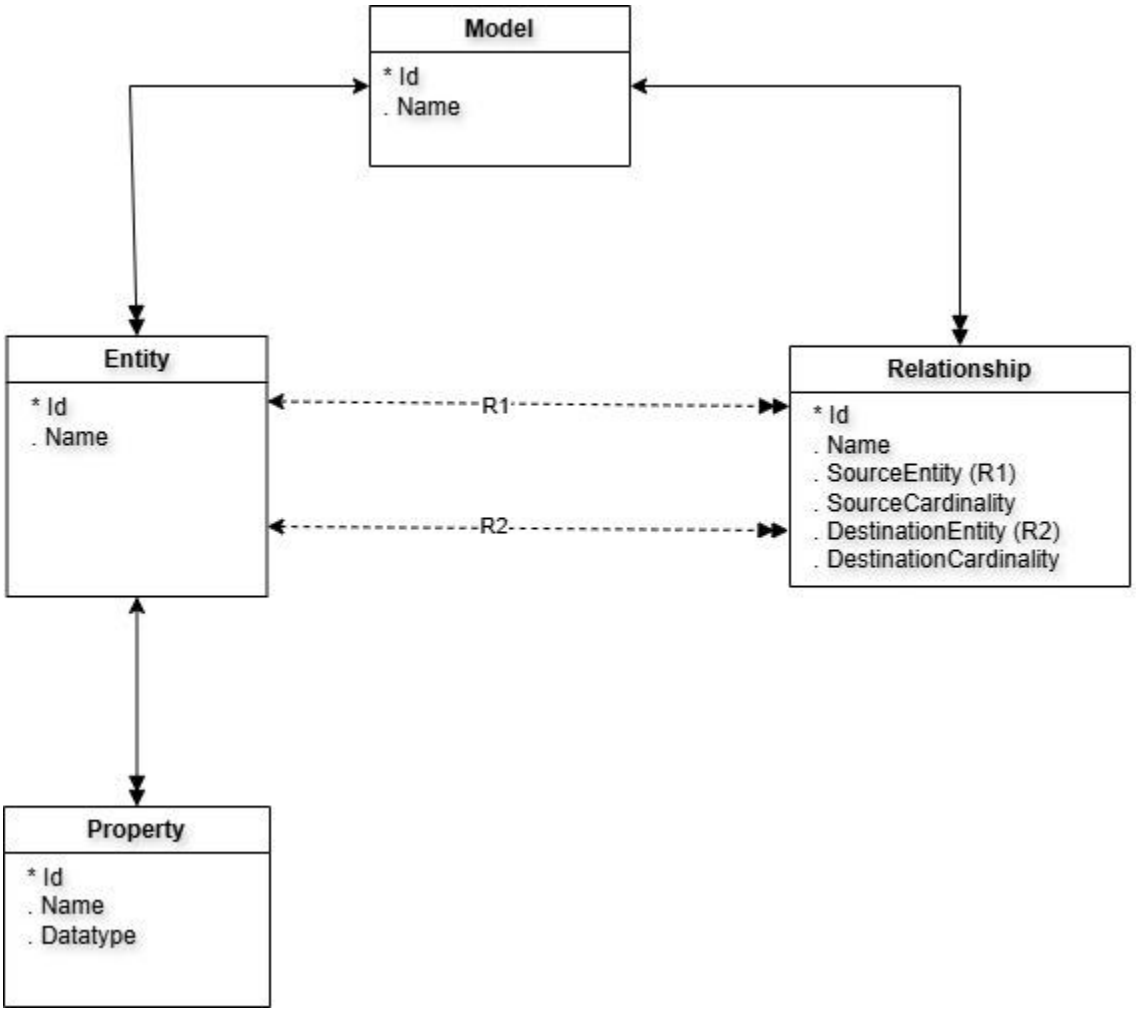
Following is an entity relationship model (in pseudo Shlaer-Mellor format) illustrating this scenario:



Again, in our scenario this model represents data, not structure. So now let's get into some model structures!

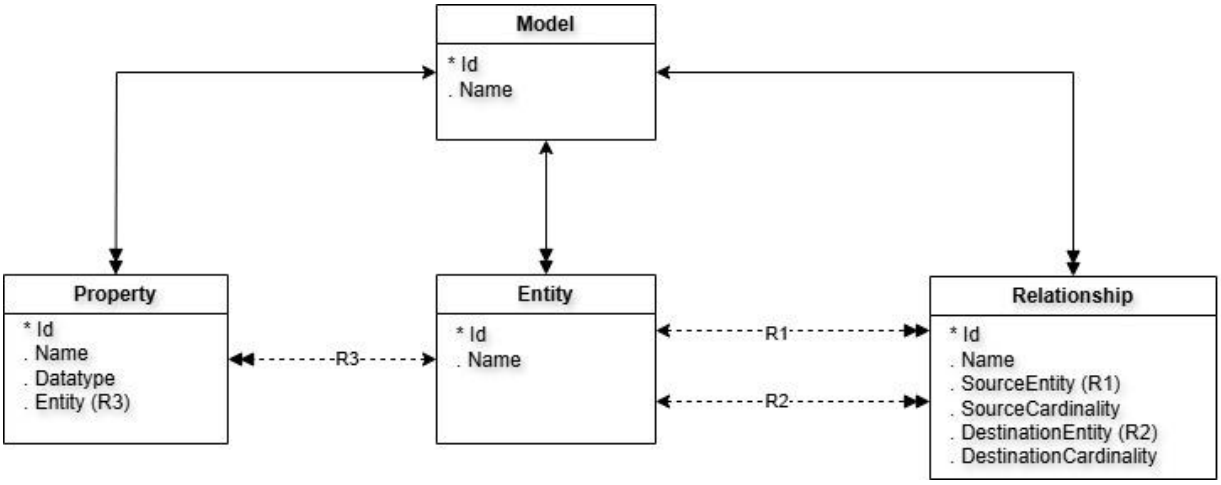
Example Model Structures

Consider the following entity relationship diagram that can represent a structure to support the model data above:



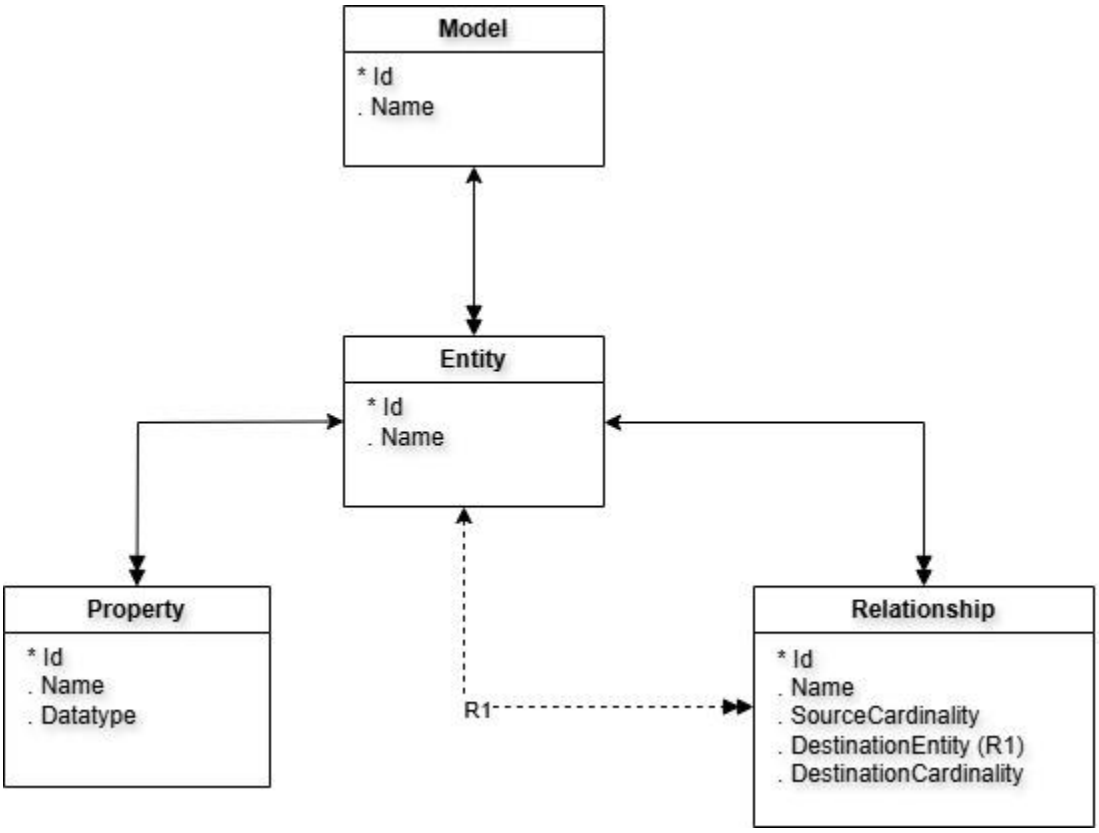
The structure has a root node of Model and child nodes of Entity and Relationship, etc. The solid lines represent the model structure hierarchy, while dashed lines alternate associations (note for these that I need to add some reference properties to “formalize” these associations). In populating this structure, instances of Entity include Restaurant, Customer, Staff, etc. and instances of Restaurant Entity properties include Id, Name, and Location, etc.

Is this the only or best structure to represent our data? Certainly not, there are many possibilities. Consider the following alternate entity-relationship model structure:



This structure puts Entity, Relationship, and Property on the same level. This might make sense if relationships and properties can exist in their own right independent of entities.

Finally consider this alternative entity relationship model structure:



In this structure, relationships and properties need an entity to exist and the relationships are children of their source entity. The reason I bring up alternative structures is this has some effect on a model oriented programming language, where hierarchies are slightly more convenient to traverse. This will be explained in more detail later.

A Model In Summary

I strongly feel that the definition of a model in terms of structure and data is an extremely important foundational principle that opens up a family of model oriented programming languages. I will explain more as we dig deeper below.

Before we continue, here are a few more definitions:

Modeling – Modeling can simply be defined as the process of populating a model structure with data.

Model Oriented Programming (MOP) – MOP can simply be defined as the process of utilizing a model to create and maintain a system (for the purpose of this article a software system).

Model Oriented Development (MOD) – MOD can simply be defined as the process of creating and utilizing a model to create and maintain a system. In other words, MOD includes Modeling and MOP.

I will use MOP and MOD acronyms in the remainder of the article for convenience.

Forms of Model Oriented Development (MOD)

To my knowledge, there are no standard names for the different forms of MOD, but I feel it is useful to describe them and their implications for MOP.

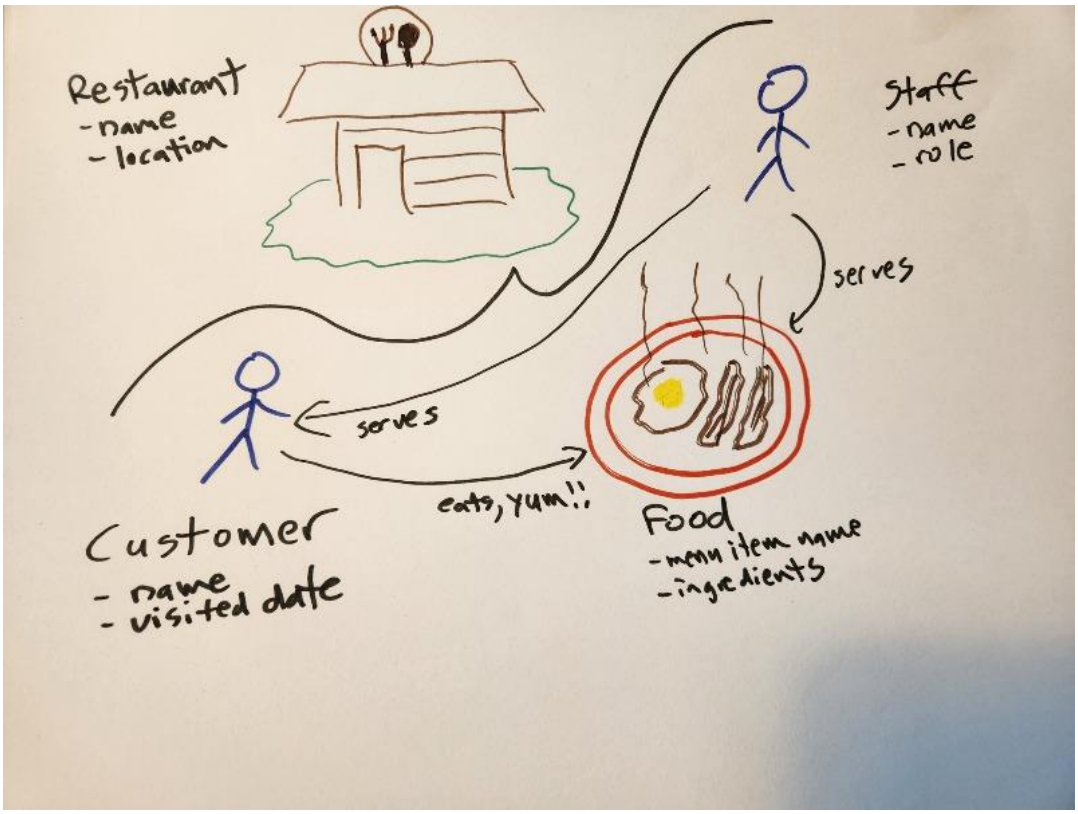
Informal MOD

Informal MOD involves utilizing any form of non-standard model. The model could be ideas in your head, chicken scratchings on a piece of paper or whiteboard, or utilizing a drawing tool. There are no rules that constrain how you create and utilize an informal model.

Informal MOD for our purposes can be roughly defined as:

- Having an ad hoc model definition outside of your source code. The models might just be ideas in your head, written descriptions, drawings, or diagrams of your own design or standard.
- There are no software tools that can utilize your model towards managing source code.

Following is a little whiteboard drawing (repeated from above) that models the basic ideas for our restaurant scenario. We will go over a potential model structure for this later on:



Inline MOD

The inline form of MOD is very popular these days with the advent and use of many frameworks that greatly enable you to effectively design and develop your applications.

Inline MOD can be roughly defined as:

- Having a formal model defined in your source code. That model is usually relevant to a particular layer in your application.
- Having a framework that typically has supporting infrastructure to allow you to manage and utilize your model, create some code for you, and/or provide infrastructure software to help manage part of your system.

Some of many examples of Inline MOD include ORMs such as Entity Framework or NHibernate and UI frameworks such as Angular and React.

Models for inline MOD are often defined in code. Below is an example Entity Framework (code first) model definition for our restaurant scenario:

```
public class Restaurant
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Location { get; set; }

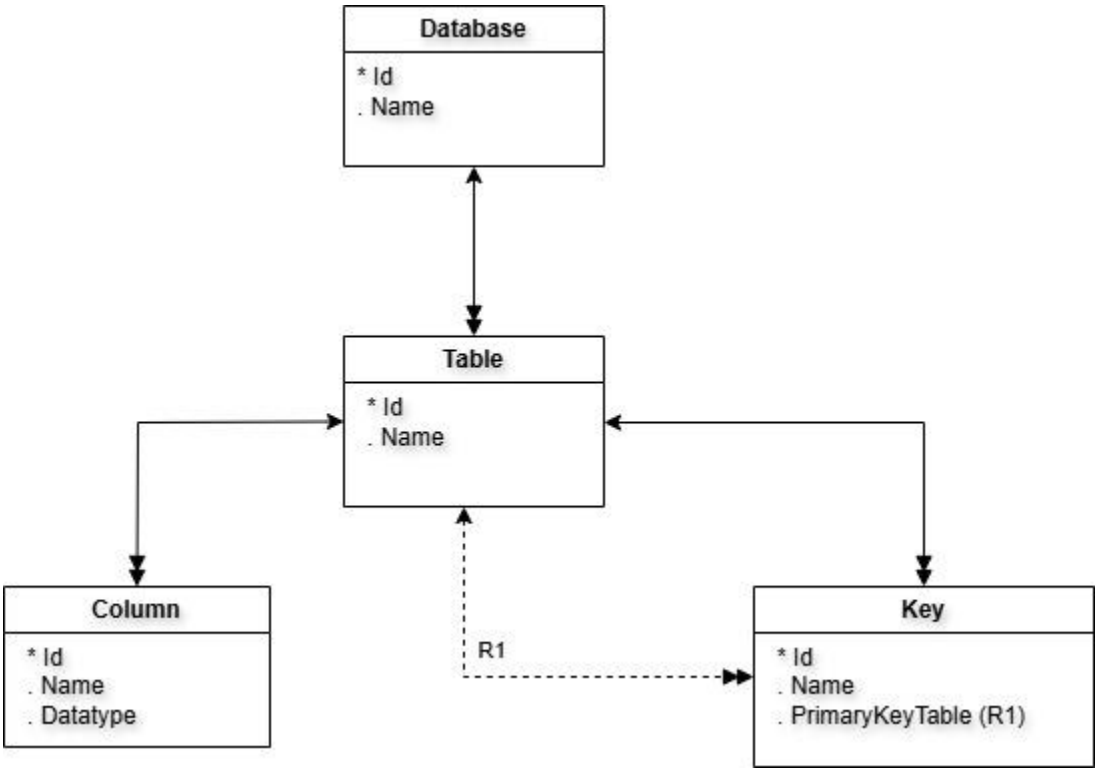
    public ICollection<Customer> Customers { get; set; }
    public ICollection<Staff> Staff { get; set; }
}
public class Staff
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Role { get; set; }

    public ICollection<Customer> CustomersServed { get; set; }
    public ICollection<Food> FoodServed { get; set; }
}
public class Customer
{
    public int Id { get; set; }
```

```
        public string Name { get; set; }
        public DateTime VisitedDate { get; set; }

        public ICollection<Food> FoodEaten { get; set; }
    }
    public class Food
    {
        public int Id { get; set; }
        public string MenuItemName { get; set; }
        public string Ingredients { get; set; }
    }
```

Any of the entity-relationship model structures presented above could be utilized to manage the above data, or a specific structure with nodes such as Table, Column, etc. could also be utilized such as the following simplified structure:



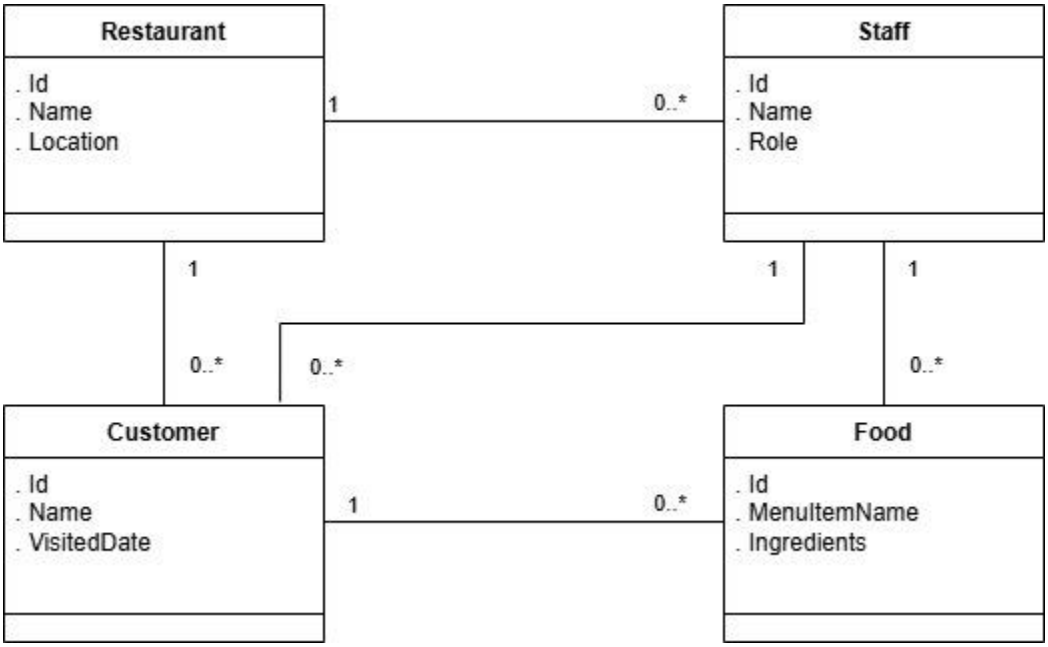
Coupled MOD

The standard UML modeling language and tools that support UML modeling and source code management with those models is the most common example of the coupled form of MOD.

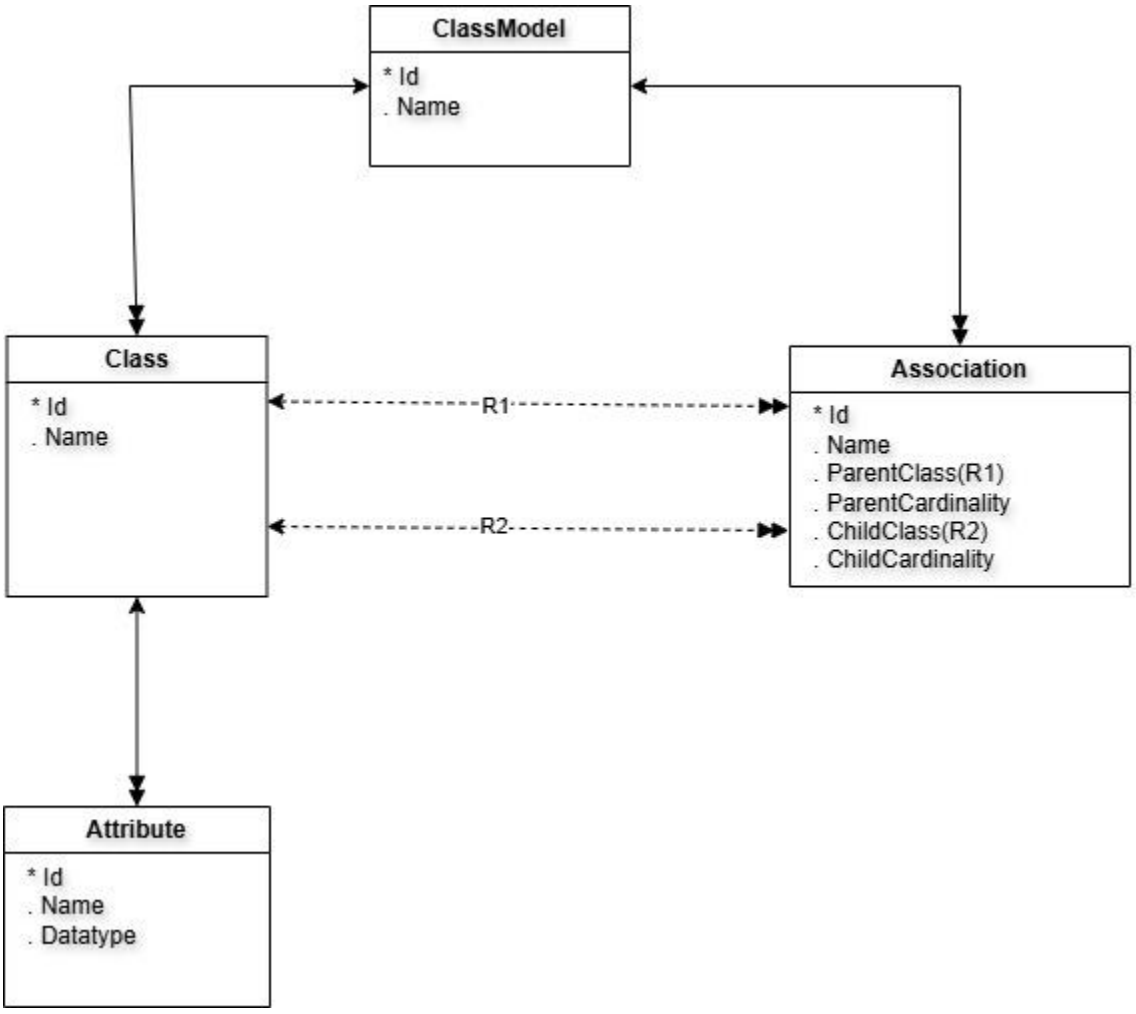
Coupled MOD can be roughly defined as:

- Having a formal model defined outside of your source code.
- There is a strong correlation (often one-to-one) between model elements and your source code. Thus, the model incorporates elements of software design.
- There are software tools that can manage your model and utilize that model for managing source code.

Many of the UML diagrams are intended to clarify the design of a system. Below is a class model diagram that outlines our restaurant scenario:



We could utilize a generic entity-relationship model structure to support class model data, but for this form of MOD, we might prefer a more design centric structure such as the following to represent a class model:



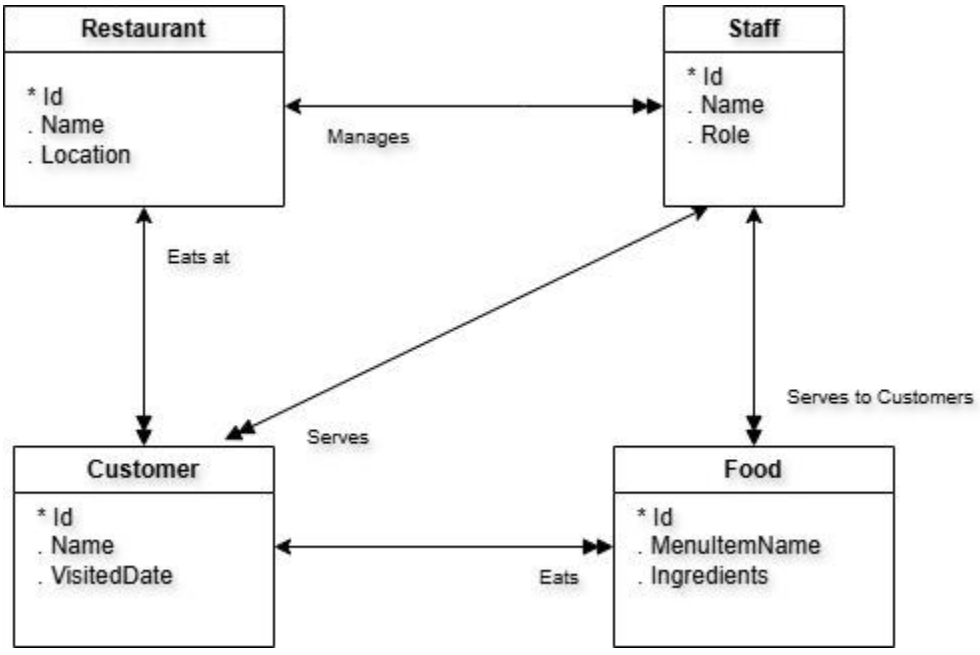
Decoupled MOD

With the heavier use of UML for specifying design, I think the decoupled form of MOD has had less attention in the industry in general. Decoupled MOD can be roughly defined as:

- Having a formal model defined outside of your source code.
- There is a weak correlation between model elements and your source code. Your source code is decoupled from the model. The abstract models tend to have very little if any design information. The detailed design is also decoupled from the model.
- There are software tools that can utilize your model for managing source code.

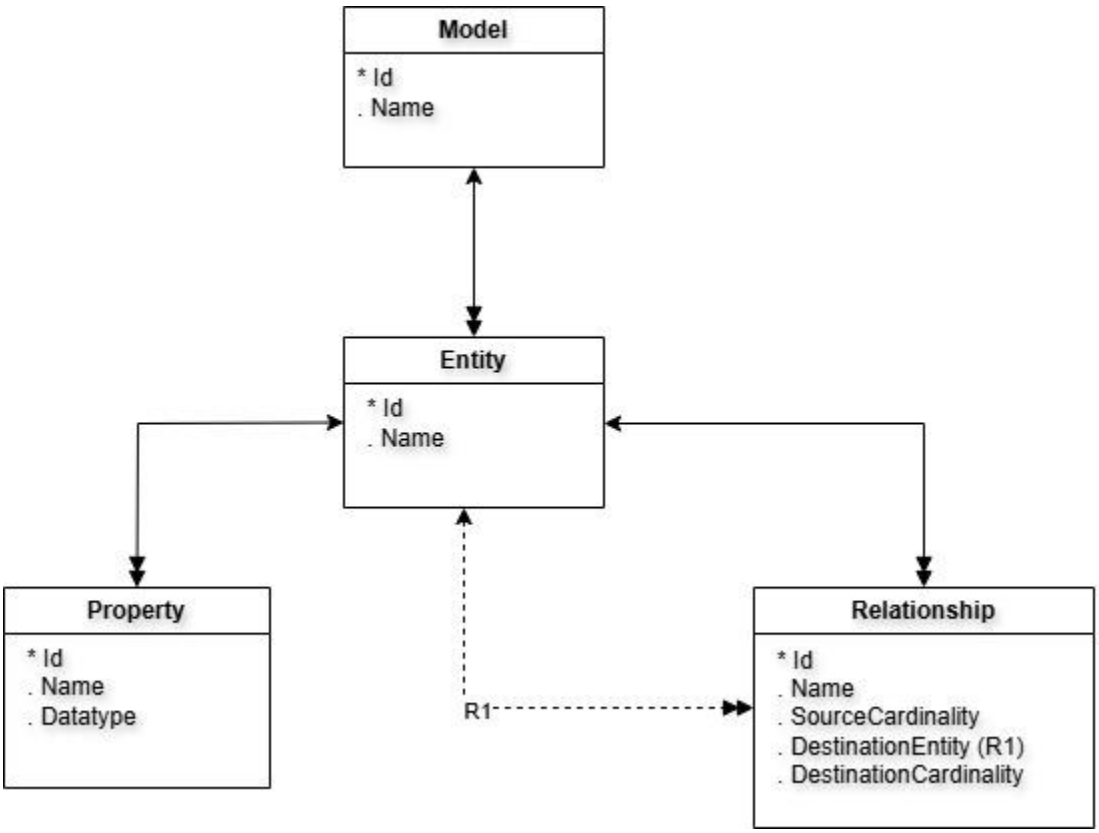
The key in recognizing this form is that the purpose of the models is to clarify ideas and requirements. Instead of modeling down to the design or component level, modeling is generally limited to requirements, primarily as data and workflow. I prefer to use the early Shlaer-Mellor method for modeling data and workflow.

Following is an information model for our restaurant scenario, repeated from above:



In decoupled MOD, elements in the model such as Restaurant don't refer to a specific component, but outline requirements for any number of related components such as tables, classes, web services, UI components, etc.

A generic entity-relationship model structure works well with decoupled MOD. As we get into coding examples below, I will utilize the following structure repeated from above:



Potential for MOP Languages by MOD Form

Can a MOP language be developed to support any of the above forms of MOD? Yes! Can a single MOP language be developed to support all of the above forms of MOD? Call me crazy, but yes! That said, I think the potential for a MOP language is least to greatest in the order below.

Potential for Informal MOD

There is potential for a MOP language to support informal MOD. The main issue is to be able to represent an informal model in an electronic form so that a language interpreter can understand the structure and data. A MOP language could indeed understand and utilize a completely nonstandard structure! I will explain how this can be done below. The benefit for supporting informal models is relatively low and probably relevant to a particular workplace or team.

Potential for Inline MOD

Speaking in terms of frameworks such as ORMs and UI frameworks, such frameworks utilize a standard model structure to do their work. So certainly a MOP language could be developed to utilize those models to further develop such frameworks and their use. The scope of the language might be limited to the scope of the framework. In addition, organizations that develop frameworks might want the MOP language only for internal development purposes.

As a side note, I implemented this to some degree years ago for the Entity Framework and VITA ORMs, which you can see in some of my Code Project articles.

Potential for Coupled MOD

With standard industry wide model structures such as UML and lots of tools to manage and utilize coupled MOD models, there is great potential here for MOP languages to be utilized at large with this form of MOD. Such languages could be utilized to create and maintain systems directly from UML models. Existing tools that translate models to code and vice versa could also be super sized with a MOP language.

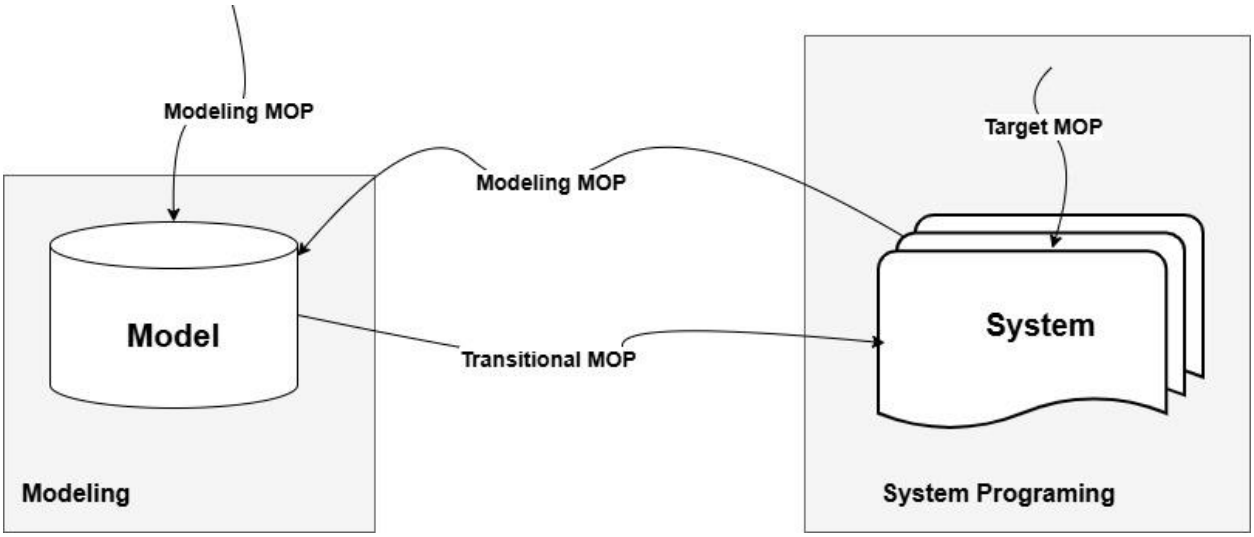
Potential for Decoupled MOD

I believe the potential for MOP languages to support decoupled MOD is even greater than coupled MOD. In addition to many industry wide benefits, putting design more in the realm of code and requirements in the realm of models makes both the modeling process and coding process more streamlined. At least that has been my personal experience.

Where to Apply Model Oriented Programming (MOP)

Before we talk about potential MOP language features, it is useful to describe different places where model oriented programming can be applied, as each of these places has different implications on MOP language needs and capabilities.

Consider the following diagram which illustrates the realms of modeling and system programming, and different places where MOP could be applied:



Transitional MOP

The purpose of transitional MOP is to interpret and utilize model structure and data towards creating and maintaining a target system environment. The target system environment could include source code in any language, documents, configuration files, etc. It is assumed that the target environment would be further enhanced with additional programming, etc. directly in the target environment.

In this case, the MOP language wouldn't need to be the language used in the target environment, but would support languages and other document standards used in the target environment. The discussion below of MOP language features will begin with this case.

Modeling MOP

The purpose of modeling MOP is to help create and maintain the structure and data of a model. Information to create and maintain the model could come from the system itself such as source code, or come from other external sources such as models, databases, documents, etc.

A MOP language to support modeling will likely need to have some features not utilized in the other cases.

Target MOP

The purpose of target MOP is to directly create and maintain a target system environment. In this case, the MOP language would be directly used in the target environment, and would likely need to be more “feature rich” than in the other cases.

MOP Language Features for Transitional MOP

Comparing with Object Oriented Languages

That was a lot to talk about above to set the stage in talking about MOP languages, I hope you are still with me! Now we can talk about various potential features of MOP languages. I will compare at a high level some differences with object oriented languages to illustrate additional potential benefits of MOP languages.

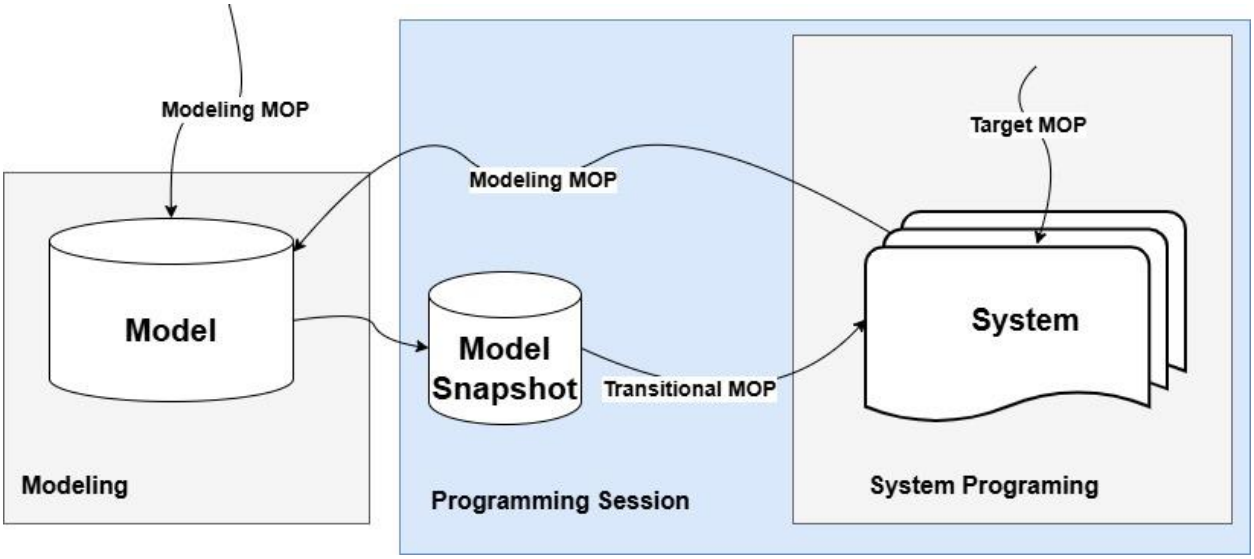
We will now start going over some simple coding examples. We will use the following colors for syntax highlighting:

- Core language grammar such as statements will be written in **blue**.
- Model structure grammar will be written in **teal**.
- Model oriented property references will be written in **brown**.
- Output will be written in **green**.

Incidentally, the language grammar presented below may or may not be similar to the language I implemented, Mo+. The grammar presented here is done for clarity.

Programming Session

For fully featured MOP languages, I think it is useful, perhaps critical, to have the notion of a programming session. To start a session, a MOP language interpreter would need to interpret and load a model structure, then load that structure with the available model data, to create a snapshot of the model as it were:



Programming Session – A period of time where a model structure is known and the model data is sealed (read only). Working within the realm of a session (however long or short) opens up the possibility of several MOP language features.

Utilizing Model Structure as Language Grammar

Knowing that a model structure is hierarchical and known at run time, and to make maximum use of such structure, it is very useful to add that structure as part of the language grammar! So, using our entity relationship example model, with the structure as part of the grammar, you can just "do stuff" with entities, relationships, properties, etc. For an object oriented language, you would have to create structures or classes for entity, relationship, property, etc. and instantiate them to have a model structure to work with. This will become clearer below when we will utilize this grammar.

Model Structure as Grammar – It is very useful to include the nodes and attributes of a hierarchical model structure into the language grammar. Adding model structure to the model grammar makes coding to the model much simpler without having to create and instantiate special classes or structures.

Utilizing Model Data as Model Context

Knowing that the model data is read only at run time and conforming to a hierarchical structure, you can imagine that it is very easy to traverse the tree of model data to get access to anything you want. It is very useful to add the notion of model context to the language grammar. Unlike object context in an object oriented language, there is no need to create specific variables or instances to hold the context. You merely need a reference to a node in the tree of model data.

Using our decoupled MOD entity-relationship structure, this is a tree view of our restaurant scenario model data:

```
Model: Name=My Model
    Entity: Name=Restaurant
        Property list: {Id, Name, Location}
        Relationship list: {Hires(Staff), Has(Customer)}
    Entity: Name=Staff
        Property list: {Id, Name, Role}
        Relationship list: {Serves(Customer), Serves to Customers(Food)}
    Entity: Name=Customer
        Property list: {Id, Name, VisitedDate}
        Relationship list: {Eats(Food)}
    Entity: Name=Food
        Property list: {Id, MenuItemName, Ingredients}
```

Let's say for example that the model context is referring to the Location property under the Restaurant entity. Consider the following statement:

```
print(Entity.Name, ".", Name)
```

The output of this statement would be Restaurant.Location. How does this work? Since the model structure is part of the grammar, and model context is on the Location property, Name yields Location. Because of the structure/data hierarchy, it is easy to get data anywhere up the tree, thus Entity.Name yields Restaurant. Going a little further:

```
print(Model.Name, ": ", Entity.Name, ".", Name)
```

Yields My Model: Restaurant.Location. Now how about accessing child data and doing more complex things?

In addition, it is useful to implement model context as a stack to enable access to the entire model tree in many different ways.

Now let's say for example that the model context is referring to the Restaurant entity. Consider the following statements:

```
foreach (Property)
{
   print(Entity.Name, ".", Name)
}
foreach (Relationship)
{
   print(Entity.Name, ".", Name, ":", DestinationEntity.Name)
}
```

This yields the following output:

```
Restaurant.Id
Restaurant.Name
Restaurant.Location
Restaurant.Hires:Staff
Restaurant.Has:Customer
```

Now what is happening here? Since the model structure is part of the grammar, it is also possible to access children. The foreach statement allows you to iterate through the children of the indicated type. Notice that there are no variables in the foreach statement. Instead, an item in the collection is pushed onto the model context stack at the beginning of the iteration and popped off at the end of the iteration statements. For example, within foreach (Property), the current model context at the bottom of the stack is the current Property in the collection. As before, you have easy access to data up the tree.

What else can you do with a model context stack? The following statements will yield the same results:

```
foreach (Property)
{
   print(../Name, ".", Name)
}
foreach (Relationship)
{
   print(../Name, ".", Name, ":", DestinationEntity.Name)
}
```

It's possible and at times very useful to access model context elements up the stack with the ../, ../ ../ notation, etc. In this case, we use this notation to get the Restaurant entity model context.

It's also possible to easily access children of children at any level in the model hierarchy:

```
foreach (Property in Model orderby Entity.Name, Property.Name)
{
   print(Entity.Name, ".", Name)
}
```

This retrieves a sorted list of all of the properties in the entire model, and yields the following output:

```
Customer.Id
Customer.Name
Customer.VisitedDate
Food.Id
Food.Ingredients
Food.MenuItemName
Staff.Id
Staff.Name
Staff.Role
```

We will do one more model context example with search. Consider the following statements:

```
with (Entity in Model where Name="Food")
{
   foreach (Relationship in Model where DestinationEntity.Name="Customer")
   {
      print(Entity.Name, ".", Name, ":", ../Name)
   }
}
```

This nested search yields the following output:

```
Restaurant.Has:Food
Server.Serves:Food
```

This last example is a little non-sensical and I'm sure there are more elegant ways to do more complex things with a model context stack, but I hope these examples give you an idea of the great variety of things you can do with model context.

Model Context – It is very useful to include the notion of model context into the language grammar. *Model Context* provides a means to access model data from a particular point in the tree. Especially if implemented as a stack to better support browsing and searching of the model data, it becomes much simpler to access overall model data without having to instantiate special context variables.

Model Oriented Properties

This is where I think it gets really interesting with MOP languages. With the model structure as grammar, I mentioned that you can just "do stuff" with entities, relationships, properties, etc. And with *Model Context* you have ready access to the tree of model data. It all comes together with model oriented properties!

Unlike object oriented properties or members that are part of a class definition, model oriented properties with the model structure as grammar become much more independent bits of code.

To begin with, each model oriented property is associated with a particular node in the model structure. Consider the following property:

```
Entity property PublicClassDeclaration
get
{
   print( "public class ", Name)
}
```

This model oriented property only works if given an instance of Entity. If given the Staff instance, the output is:

```
public class Staff
```

For this article I will use print and println to build up content within a property get method that will subsequently be returned. Now let's define a similar Property level model oriented property:

```
Property property PublicPropertyDeclaration
get
{
   print( "public ", Datatype, " ", Name, " {get; set;}")
}
```

This model oriented property only works if given an instance of Property. If given the Staff.Role instance, the output is:

```
public string Role {get; set;}
```

Model oriented properties can utilize other model oriented properties. Consider the following:

```
Entity property PublicClassCode
get
{
   println( PublicClassDeclaration)
   println("{")
   foreach (Property)
   {
      println("    ", PublicPropertyDeclaration)
   }
   println("}")
}
```

If given the Staff instance, the output is:

```
public class Staff
{
   public int Id {get; set}
   public string Name {get; set}
   public string Role {get; set}
}
```

Finally, since model oriented properties are independent bits of code, they can be used in many different ways such as part of expressions:

```
foreach (Entity in Model where PublicClassCode.Contains("Role") == true)
{
   print(Name)
}
```

This silly example finds all entities that contain "Role" in the class code, which finds the Staff entity:

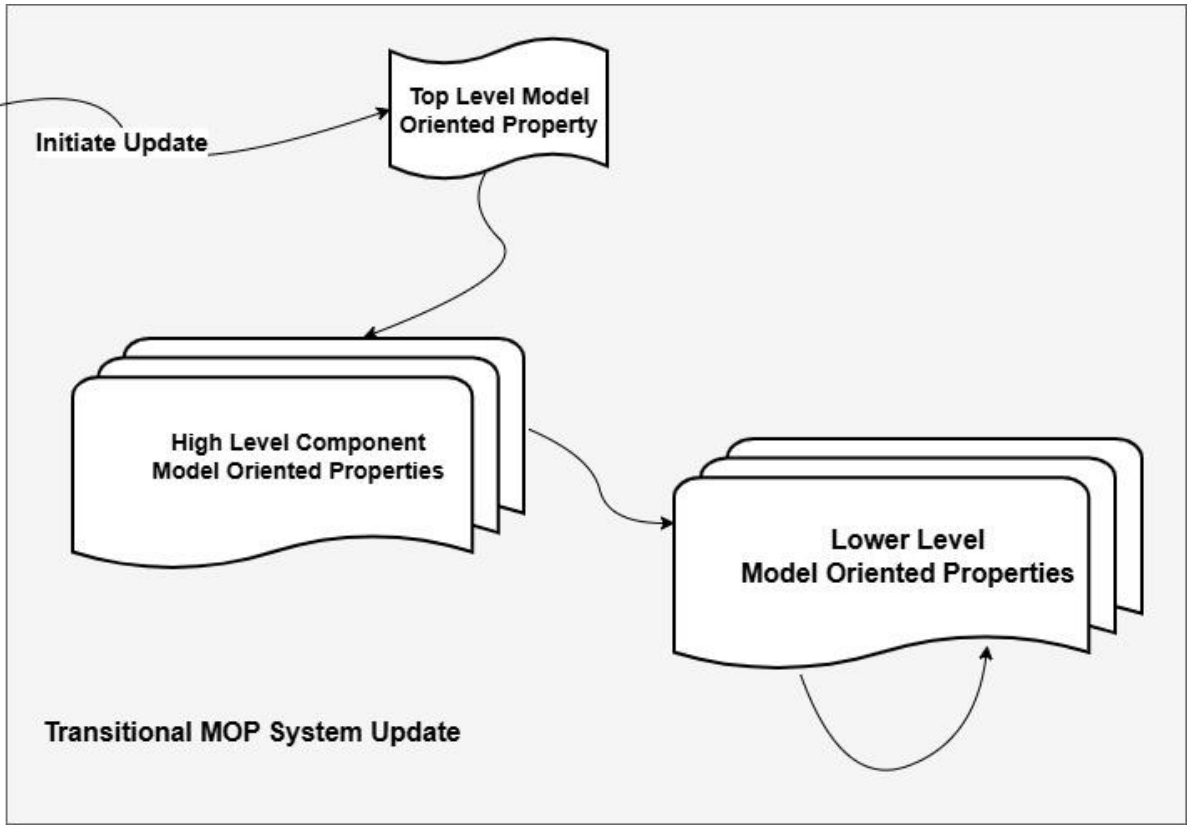
```
Staff
```

What other methods can model oriented properties have? For transitional MOP where the model is sealed in a session, it doesn't make sense to have a set method to update the model. But a put method makes sense to add and update information in the target system. Here, a put method is added to the PublicClassCode method:

```
Entity property PublicClassCode
get
{
   println( PublicClassDeclaration)
   println("{")
   foreach (Property)
   {
      println("    ", PublicPropertyDeclaration)
   }
   println("}")
}
put
{
   save(content,  PublicClassFilePath)
}
```

The put method saves the class code to a file location specified by the PublicClassFilePath property. The content keyword is a shorthand for getting the results from the get method.

To create or maintain a system, essentially a “bag” of model oriented properties can be chosen that meets the technical requirements for that system. A program run is initiated with a top (model) level property that in turn initiates high level component (project) properties which in turn initiate lower level properties, etc.



With the ability to define and use model oriented properties without instantiating classes or containers, the ability to utilize model oriented properties as building blocks for much more complex properties, and the ability to freely utilize model oriented properties in expressions, I hope you get the picture on how versatile these properties can be.

Model Oriented Property – A model oriented property is an independent bit of code associated with a node in the model structure (such as Entity). At run time, a model oriented property can be evaluated when given an instance of the same node (such as Entity) as the Model Context. A model oriented property can be evaluated many different times with different instances of the same node, and can be utilized in many more ways than object oriented properties.

Dynamic Grammar

A dynamic grammar for a programming language is a grammar that can change or be augmented at runtime. Do any other programming languages out there have a dynamic grammar?

If a model structure is hierarchical and is part of the language grammar, it is possible and actually useful for a MOP language to have a dynamic grammar. It is actually very straightforward for a MOP language interpreter to implement a grammar builder that reads in a model structure and add that structure to the grammar at the beginning of a session. It's just a matter of adding nodes and attributes to the model hierarchy grammar. I will cover some examples of extending a model hierarchy in the Modeling MOP section below.

Now why would a dynamic grammar be useful? I will give a couple of examples.

Insufficient Model

Even with a standardized model, I often find that the standard model sometimes lacks some information that I need to have to effectively program against it. There is a cheap way to add data to nodes in the form of tags that is described in more detail below, but sometimes this is not sufficient, and we may need to add more attributes to nodes or even new nodes to the model structure.

In our case we decided we need to add the *MaxLength* attribute to the *Property* node in our model structure. When the grammar builder recognizes and adds this attribute to the grammar, it can be used just like any other attribute of *Property*.

Here is a new property and updated class code property to make use of MaxLength:

```
Property property StringLengthAttribute
get
{
   if ( MaxLength > 0)
   {
      print(" [StringLength(", MaxLength, ")]")
   }
}

Entity property PublicClassCode
get
{
   println( PublicClassDeclaration)
   println("{")
   foreach (Property)
   {
      if ( StringLengthAttribute != "")
      {
         println("    ", StringLengthAttribute)
      }
      println("    ", PublicPropertyDeclaration)
   }
   println("}")
}
put
{
   save(content,  PublicClassFilePath)
}
```

And the updated class code for Staff looks like (MaxLength only set on Name):

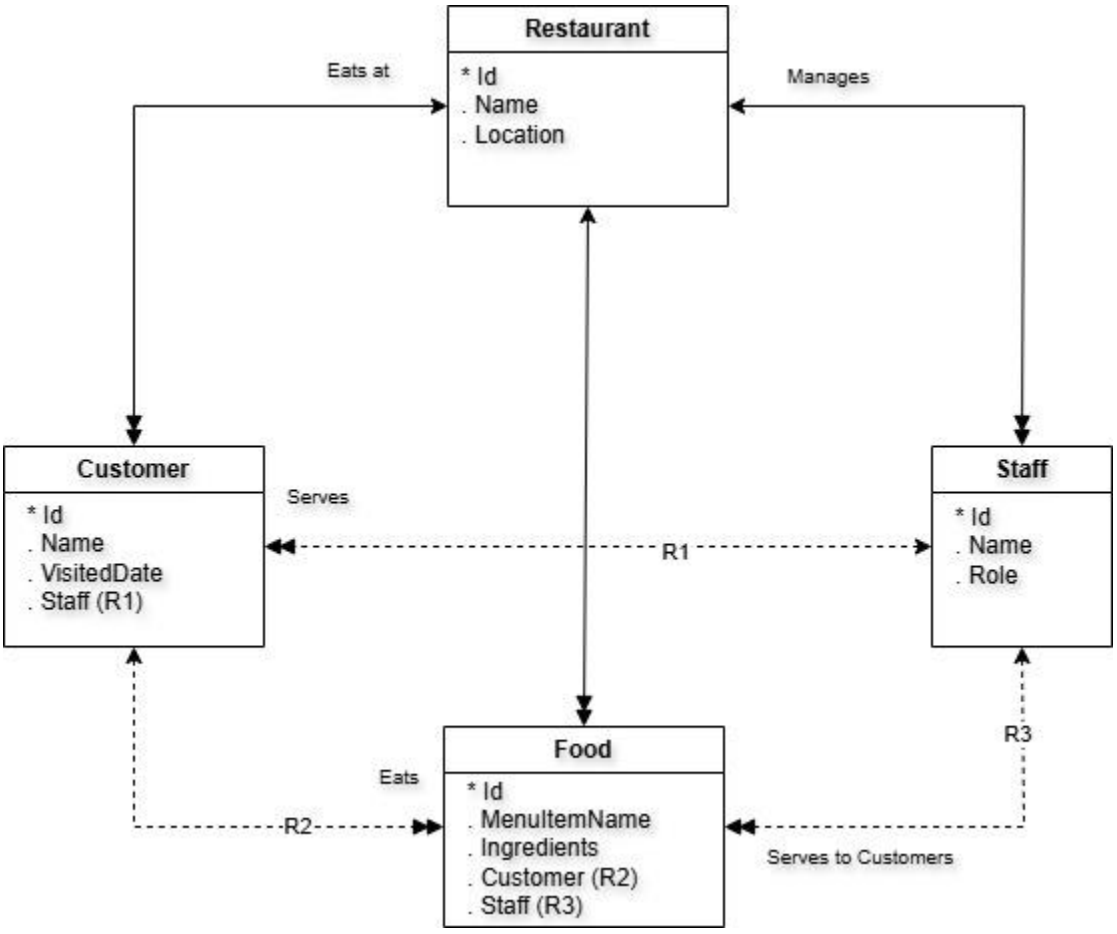
```
public class Staff
{
   public int Id {get; set}
   [StringLength(255)]
   public string Name {get; set}
   public string Role {get; set}
}
```

Non standard Model

Now what if a model structure is partially or completely non-standard or otherwise specific to an organization or industry? We would need to interpret and build the grammar for the whole model structure. Again, the grammar builder could easily do that.

A model for informal MOD would be non-standard, and having a dynamic grammar to load informal model structure and data is the only way that Informal MOD can be effectively supported.

Let's do something completely different. Using the informal drawing of the restaurant scenario above, here is a more literal model structure:



This structure probably wouldn't make sense if we were building an enterprise application, but if this is the structure, what is the data? It would be the same as the run time data! So in effect our language could become a query language. Here is an example query:

```
foreach (Restaurant)
{
   println(Name, ":   ", Location)
   foreach (Staff)
   {
      println("    Staff: ", Name, ",   ", Role)
   }
   foreach (Customer)
   {
      println("    Customer: ", Name)
   }
}
```

Some query output might look like:

```
Chez Food: London
   Staff: Verity Jones, Chef
   Staff: Nigel Walker, Server
   Customer: David Spencer
   Customer: Catherine Rees
Chez Food: Paris
   Staff: Monique Dubois, Chef
   Staff: Henrique Bernard, Server
   Staff: Michelle Fontaine, Server
   Customer: Pierre Simon
   Customer: Camille Fournier
```

I haven't tried reading in a literal structure to do queries, so this is pure conjecture. It wouldn't be practical to load lots of runtime data into a session.

Perhaps another reason to have very specific elements in your model structure such as Restaurant may be to allow you to write very specific model oriented code for an element that is much different than others. In practice, I have only found the need for a generic structure.

Grammar Builder – A language interpreter mechanism that modifies or augments the language grammar at runtime. A MOP language interpreter can read in a model structure and add that structure to the grammar at the beginning of a session.

Contextual Grammar

Perhaps not as important as dynamic grammar, but I think the concept of contextual grammar is useful for MOP languages, especially since MOP can be applied in different places such as to support modeling, transitioning from a model, and target programming. As with a dynamic grammar, it is very straightforward for a MOP language interpreter to implement a grammar filter to make the appropriate grammar available by context to help prevent coding that can cause unintended consequences.

Globally

For certain model oriented property methods, I think it is useful to limit the ability of what can be coded to enforce division of responsibility and help prevent unintended consequences. We may want filters such as:

- Within model oriented property ***get*** methods, inhibit the ability to write information to other places in any way. Filter out language grammar to prevent statements that can write data anywhere.
- Within model oriented property ***put*** methods, allow the ability to write information to other places other than the model. Filter out language grammar to prevent statements that can write data outside of models.
- Within model oriented property ***set*** methods, allow the ability to write information only to the model. Filter out language grammar to prevent statements that can write data to models.

Transitional MOP Filter

For programming to support utilizing a model to manage a target environment, we may want filters such as:

- Inhibit the ability to create ***set*** methods in model oriented properties that are used to update models and invalidate programming sessions.

Modeling MOP Filter

For programming to manage a model, we may want filters such as:

- Inhibit the ability to create ***put*** methods in model oriented properties that are used to update information outside of models.

Target MOP Filter

For programming to manage a target environment directly, there probably would be a more robust grammar to support anything target programmers may need to do. Perhaps in this case it is desired to allow all types of model oriented property methods with the appropriate grammar filters within them, knowing that ***set*** methods can invalidate the programming session.

Grammar Filter – A language interpreter mechanism that filters the overall language grammar based on context to help prevent coding that can cause unintended consequences.

Almost No Variables!

Transitional MOP requires very few variables! With Model Context and having access to the entire tree of model data, there are many cases where variables are not needed where they are required in other languages:

- No variables are required for holding iterations when searching or browsing.
- No variables are required to evaluate model oriented properties.
- Model oriented properties can be utilized in many different places such as part of expressions.

In fact, I only implemented support for variables very late in the process. For transitional MOP, aside from a handful of global (state) settings, I only used variables for maybe 1% of model oriented properties I developed at most, and perhaps all of those cases could have been handled with better model oriented programming or augmenting the model structure a bit. I found that I almost always could write a model oriented property to express what I would do in another language and hold in variables.

Massive Reusability

As mentioned above, model oriented properties are extremely flexible, being independent bits of code that utilized almost anywhere. This flexibility enables massive reusability, both within a program and across programs. I have found in practice that model oriented properties are often essentially codified best practices that transcend functional requirements of any given project.

The following is a very simple example of a class naming convention that can be reused hundreds of times. Of course much more complex "best practice" properties can be reused time and time again.

```
Entity property ClassName
get
{
   print( Name.PascalCase())
}
```

Massive Parallelism

If almost no variables are required and the model data is sealed in read only in a programming session, you can infer that there is potential for massive parallelism in evaluating model oriented properties.

I didn't implement parallelism, but I did implement caching. The interpreter would cache certain results of evaluating a model oriented property for a particular model context (instance of a node), so that those results could be utilized many times later. I found great performance gains in just doing this.

Integration with Target Language (Custom) Code

My experience with Transitional MOP is that with a reasonable amount of effort you can manage 80% of a target enterprise environment to best practices, and maybe with double or more the effort, you could get to 90%. In any case, you need to have an approach to be able to maintain the target environment 100% to best practices, especially when making updates with Transitional MOP.

This is an area that requires more thought and sophistication. None of the following is ideal, but here are a few things that can be done:

- **Partial files** – The easiest starting point is to have files created and updated by the MOP language interpreter is to keep them separate from files created or modified in the target language directly. Partial class files are an example of this.
- **Customization points** – Partial files often only go so far to support customization. The ability to define flexible customization points (such as named regions or comments) within files managed by the interpreter. Thus, custom code can be added within these points, and the interpreter can reapply the custom code if the overall file needs to be updated.
- **File update and deletion policy** – The interpreter should only update and delete files it manages, and should leave other custom files alone. In addition, if there is any custom code within managed files, those should not be deleted.
- **Ignore policy** – Finally, if any files that were managed by the interpreter need to be heavily customized, they can be marked as ignore so that the interpreter in the future will leave them alone.

Implications by MOD Form

All of the above features could be utilized in each of the forms of MOD, with some possible caveats:

Programming Session – Programming sessions would likely get stale more often with Coupled MOD, as design changes and enhancements happen very frequently. In addition, since detailed design models are much larger than for other forms, more memory would be required to load the model data.

Dynamic Grammar – Having a dynamic grammar and a grammar builder would be a requirement for Informal MOD, as there is no model structure standard.

Massive Reusability – This is more of a hunch than empirical data, but I feel that there would be greater usability when utilized with Decoupled MOD. With this form, I found that most of my model oriented properties were essentially codified best practices that I could reuse many times within and across many projects.

Implications for Modeling MOP

As the purpose of modeling MOP is to create and maintain the structure and data of a model, some requirements for a MOP language differ than that of Transitional MOP. Creating and maintaining a model structure and data of course can be done manually with an IDE, but here are also some desirable features to help support modeling programmatically:

Defining Model Structure

Being able to define or augment a model structure I think is a critical feature for a MOP language in order to handle many programming scenarios. Basic features include being able to add and edit nodes in a hierarchical model structure, and being able to add and edit attributes to these nodes. In turn, this information can be passed on to the grammar builder to add the structure to the model grammar.

Below is an example of simple statements to create our restaurant model structure with Restaurant as the root node:

```
createModel ("Restaurant")
addNode (Restaurant, "Customer")
addNode (Restaurant, "Staff")
addNode (Restaurant, "Food")
addAttribute (Restaurant, "Id")
addAttribute (Restaurant, "Name")
addAttribute (Restaurant, "Location")
addAttribute (Staff, "Id")
addAttribute (Staff, "Name")
addAttribute (Staff, "Role")
addAttribute (Customer, "Id")
addAttribute (Customer, "Name")
addAttribute (Customer, "VisitedDate")
addAttribute (Food, "Id")
addAttribute (Food, "MenuItemName")
addAttribute (Food, "Ingredients")
```

Notice that statements to add or edit model structure can immediately update the grammar with the grammar builder so that new model elements can be utilized right away. A complete statement set to manage model structures would contain more features such as edit and delete and more information such as data types, etc.

Import/Export

An important feature for supporting modeling MOP is to support import/export with other sources of model data. Some examples include:

- **XML documents** – XML import/export requires being able to determine the structure of the input XML and how nodes and attributes in the XML correspond to the model structure being used in order to perform the import of model data.
- **Databases** – Database import requires being able to set up connection strings for supported database types, being able to read the schema for a database, and how the schema elements correspond to the model structure being used in order to perform the import of model data.
- **Other models** – Similar to the above examples, model import/export requires being able to understand corresponding model structures in order to perform the import or export of model data.

In actual practice, I utilized import from other model sources for the majority of my modeling work for enterprise projects. If you are doing manual modeling as well as import, it is important for the IDE to keep track and maintain manual changes to the model when subsequent imports of model data occur.

Tagging

Tagging is a simple way of adding some unstructured data to any element in the tree of model data as a way of specifying additional requirements. The tags could just be a list of string values or name/value pairs attached to a model element to differentiate it from other elements.

Say for example that staff names are confidential and we have an attribute to encrypt a property. We can add a tag to the Staff Name property such as “secure”, and that tag can be utilized to handle the Staff Name property differently:

```
Entity property PublicClassCode
get
{
   println( PublicClassDeclaration)
   println("{")
   foreach (Property)
   {
      if (Tags.Contains("secure"))
      {
         println("    [Encrpyt]")
      }
      println("    ", PublicPropertyDeclaration)
   }
   println("}")
}
```

And the class code for Staff looks like:

```
public class Staff
{
   public int Id {get; set}
   [Encrypt]
   public string Name {get; set}
   public string Role {get; set}
}
```

Implications for Target MOP

Since I did not implement a MOP language as a target language, this section is pure conjecture on my part and would be a main area for ongoing research.

Language Grammar

A target MOP language would undoubtedly need a more extensive grammar than that for Transitional MOP to handle all of the things that a target language programmer would need to do. Amongst other things, type safety is not necessary for Transitional MOP but would probably be essential for target MOP.

Integration with Transitional MOP

I think this is a very interesting area and would require much thought on how to manage this integration. I mentioned above with Transitional MOP that you could define customization points to manage and maintain custom code. I think for a true target MOP language, we would need to be much smarter than that.

Let's imagine that Transitional MOP is used to utilize the model and build and maintain the basic or even very sophisticated enterprise framework for your project. How will you add your target code to this framework? If updates happen to the framework based on Transitional MOP (best practice or other) changes, how will you be able to manage changes to your target code? I don't have any specific ideas on this, but I can at a high level imagine a smart IDE that allows the programmer to view updates to the framework vs target code and help with the help with the process of updating target code. What do you think?

Integration with Modeling MOP

Modeling MOP or manually updating the model data would have the same implications for target programming as above. When a session and the framework is updated, as with above, I think we would need a smart and graphical IDE to help manage changes to the target code.

Now, what happens when we change the model **structure** manually or through Modeling MOP? This would change the MOP language grammar! How would this be managed? Hmmm!!

With Transitional MOP, deletion of elements of the model structure would create compilation errors that can be resolved and addition of model structure would only be utilized in model oriented property changes.

To manage target code when the language grammar changes, again I think we would need a smart IDE to manage this. Exactly how, I have no great ideas. What do you think?

Variables

Certainly as a target language, there would be more of a need for variables to handle any kind of user or other I/O. Even so, I believe that variable use would be much lower than with other languages.

New Language or Supersize an Object Oriented Language?

As a target language, I think there might be a significant overlap in the language grammar with an object oriented language. So, should a MOP language as a target language be a separate language in its own right, or an enhancement to an existing object oriented language? Food for thought!

In Conclusion

I hope this article helps inspire looking into the potential of model oriented languages. There are a lot of concepts to digest here, and please review and let me know how I can make this more clear in any way, and if there is any related work out there that I can refer to. I will happily update this article with more feedback.

Several years ago, I wrote a number of articles related to model oriented languages and development on codeproject.com. I hope the site become active again in the future.

If you want to explore the language and IDE I developed years ago, Mo+, it is still available on github.com/moplus. It was last available as a Visual Studio add-on in 2014 but the source code and standalone installer is still available. The IDE includes a full help system and online reference to try out the language. The project includes templates (packages of model oriented properties) that you can use to create some sample projects.